Name: Yash kr. Srivastava
CU ID: CU24250007
Roll No.: 70

EXPERIMENT NO. 7

Customer Segmentation using K-Means Clustering

Step 1 — Import libraries and load the dataset

```
import numpy as np import
pandas as pd
import matplotlib.pyplot as plt import seaborn as
sns

from sklearn.preprocessing import StandardScaler from
sklearn.cluster import KMeans from sklearn.metrics import
silhouette_score

sns.set(style="whitegrid")

# Load the dataset
df = pd.read_csv("Mall_Customers.csv")

print("Shape of dataset :", df.shape) print(df.head())
```

Output:

```
Shape of dataset : (200, 5)

   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40
```

Step 2 — Data exploration and preprocessing

```
# Structure and summary df.info()
print(df.describe())

# Check for missing values and duplicates print("Missing values:\n", df.isnull().sum())
print("Duplicate rows :", df.duplicated().sum())

# Drop duplicates if any and remove the identifier column df = df.drop_duplicates()
df = df.rename(columns={"Annual Income (k$)": "Income",                    "Spending Score (1-100)":
"Score"})
```

```
# Encode Gender (useful for later profiling)
df["Gender_encoded"] = df["Gender"].map({"Male": 0, "Female": 1})
```

Output:

```
Missing values:
 CustomerID              0
 Gender                  0
 Age                     0
 Annual Income (k$)      0
 Spending Score (1-100)  0
Duplicate rows : 0
```

The dataset is clean — there are no missing values and no duplicate records, so no imputation is required.

Step 3 — Feature selection and scaling

Annual Income and Spending Score are selected because they directly describe purchasing power and purchasing behaviour, and because two features allow the clusters to be visualised on a plane. The features are standardised so that the larger numeric range of income does not dominate the Euclidean distance calculation.

```
X = df[["Income", "Score"]].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Mean after scaling :", np.round(X_scaled.mean(axis=0), 3)) print("Std  after scaling :",
np.round(X_scaled.std(axis=0), 3))
```

Output:

```
Mean after scaling : [ 0. -0.] Std  after scaling
: [1. 1.]
```

Step 4 — Elbow Method to find the optimal K

```
wcss = []
K_range = range(1, 11)

for k in K_range:
    km = KMeans(n_clusters=k, init="k-means++",                n_init=10,
max_iter=300, random_state=42)     km.fit(X_scaled)
wcss.append(km.inertia_)

plt.figure(figsize=(7, 4)) plt.plot(K_range, wcss, marker="o",
linestyle="--") plt.title("Elbow Method for Optimal K")
plt.xlabel("Number of clusters (K)") plt.ylabel("WCSS (Inertia)")
plt.xticks(K_range) plt.show()
```

Observed WCSS values:

K	WCSS (Inertia)	K	WCSS (Inertia)
1	398.00	6	55.14
2	269.69	7	44.61
3	157.42	8	36.88
4	108.92	9	32.17
5	65.57	10	28.94

The curve falls steeply up to K = 5 and then flattens noticeably. The bend of the curve — the elbow — therefore occurs at K = 5.

Step 5 — Validation using the Silhouette Score

```
sil_scores = {}

for k in range(2, 9):
    km = KMeans(n_clusters=k, init="k-means++",
n  init=10, random  state=42)     labels = km.fit  predict(X  scaled)
    sil_scores[k] = silhouette_score(X_scaled, labels)
    print(f"K = {k}  ->  Silhouette Score = {sil_scores[k]:.4f}")

best_k = max(sil_scores, key=sil_scores.get) print("Best K by silhouette
score :", best_k)
```

Output:

Number of clusters (K)	Average Silhouette Score	Remark
2	0.2949	Poor separation
3	0.4671	Improving
4	0.4938	Good
5	0.5546	Best — well separated
6	0.5398	Slightly lower
7	0.5262	Decreasing
8	0.4554	Clusters begin to overlap

The highest average silhouette score (0.5546) is obtained at K = 5, which confirms the result of the Elbow Method. Hence K = 5 is selected.

Step 6 — Applying K-Means with K = 5

```
kmeans = KMeans(n_clusters=5, init="k-means++",                n_init=10,
max_iter=300, random_state=42)

df["Cluster"] = kmeans.fit_predict(X_scaled)

# Centroids brought back to the original units
centroids = scaler.inverse_transform(kmeans.cluster_centers_)
print(pd.DataFrame(centroids, columns=["Income (k$)", "Spending Score"])) print("\nCustomers per cluster:\n",
df["Cluster"].value_counts().sort_index())
```

Step 7 — Visualising the clusters

```
plt.figure(figsize=(8, 6))
colors = ["red", "blue", "green", "cyan", "magenta"]
names  = ["Careful", "Standard", "Target", "Careless", "Sensible"]

for i in range(5):
    plt.scatter(X[df["Cluster"] == i, 0], X[df["Cluster"] == i, 1],                s=60, c=colors[i],
label=f"Cluster {i+1} - {names[i]}")

# Highlight the centroids
plt.scatter(centroids[:, 0], centroids[:, 1],             s=300, c="yellow",
marker="*",             edgecolors="black", label="Centroids")

plt.title("Customer Segments (K-Means, K = 5)")
plt.xlabel("Annual Income (k$)") plt.ylabel("Spending Score
(1-100)") plt.legend() plt.show()
```

The scatter plot shows five clearly separated groups arranged roughly in the shape of a cross: a dense central group of average customers, and four corner groups representing the extreme combinations of high/low income with high/low spending. The yellow stars mark the centroid of each segment.

Step 8 — Profiling each segment

```
profile = df.groupby("Cluster").agg(
    Customers      = ("CustomerID", "count"),
    Avg_Age        = ("Age", "mean"),
    Avg_Income     = ("Income", "mean"),
    Avg_Score      = ("Score", "mean"),
    Female_Percent = ("Gender_encoded", lambda s: 100 * s.mean()) ).round(1)

print(profile)
```

Results and Analysis

Cluster profile

Cluster	Customers	Avg. Age	Avg. Income (k$)	Avg. Spending Score	Interpretation
0	23	45.2	26.3	20.9	Low income, low spending

1	81	42.7	55.3	49.5	Average income, average spending
2	39	32.7	86.5	82.1	High income, high spending
3	22	25.3	25.7	79.4	Low income, high spending
4	35	41.1	88.2	17.1	High income, low spending

Business interpretation and marketing strategy

Segment	Label	Behaviour	Recommended marketing strategy
Cluster 2	Target / Premium Customers	Earn the most and spend the most; the most profitable group.	Loyalty and VIP programmes, premium and luxury brands, early access to new collections, personalised offers. Highest priority for retention.
Cluster 3	Careless / Impulsive Buyers	Low income but very high spending; mostly young customers.	EMI and buy-now-pay-later options, seasonal sales, trendy affordable products, social-media and influencer campaigns.
Cluster 4	Sensible / Cautious Customers	High income but very low spending; large untapped potential.	Quality- and value-focused messaging, product demonstrations, warranties, exclusive previews and surveys to understand why they do not spend.
Cluster 1	Standard Customers	Average on both income and spending; the largest group.	Combo offers, cross-selling, reward points and membership cards to gradually move them towards the premium segment.
Cluster 0	Careful / Budget Customers	Low income and low spending; least profitable.	Discount coupons, budget product lines and essential goods. Keep acquisition cost low; avoid heavy marketing spend.

Observations

- Both the Elbow Method and the Silhouette Score agreed on K = 5, which increases confidence in the choice.
- Annual income alone does not predict spending — Clusters 3 and 4 show that a low-income customer may spend heavily while a high-income customer may spend very little.
- The average age of the high-spending segments (Clusters 2 and 3) is considerably lower than that of the low-spending segments, indicating that younger customers spend more freely.
- About 40% of the customers fall in the average-average segment, which represents the biggest opportunity for upselling.
- Feature scaling changed the cluster boundaries slightly compared with unscaled data, confirming that KMeans is sensitive to the scale of the input features.

Conclusion

The K-Means clustering algorithm was successfully implemented on the Mall Customer Segmentation Dataset. After preprocessing and standardising the features, the optimal number of clusters was determined to be five using the Elbow Method and independently validated by the Silhouette Score (0.5546). The five resulting segments — Premium, Impulsive, Cautious, Standard and Budget customers — were clearly separated and could be interpreted in meaningful business terms. The experiment demonstrates that customer segmentation through unsupervised learning enables an organisation to replace a single generic marketing strategy with a set of focused strategies, thereby improving campaign effectiveness, customer retention and overall profitability.

Viva Questions and Answers

Q1. What is clustering? How does it differ from classification?

Clustering is an unsupervised learning technique that divides a set of unlabelled data points into groups, such that points within a group are similar to each other and dissimilar to points in other groups. The algorithm discovers the structure in the data on its own.

Difference from classification:

Basis	Clustering	Classification
Learning type	Unsupervised	Supervised
Data required	Unlabelled data	Labelled training data
Goal	Discover natural groupings	Predict a predefined class label
Output	Cluster numbers with no inherent meaning	A known class for each record
Evaluation	Silhouette score, Davies–Bouldin index, inertia	Accuracy, precision, recall, F1-score
Example	Grouping mall customers by spending habits	Classifying an email as spam or not spam

Q2. Explain the working principle of the K-Means Clustering algorithm.

K-Means is a centroid-based algorithm that partitions n observations into K clusters by minimising the withincluster sum of squares. Its working steps are:

1. Choose the number of clusters K.
2. Initialise K centroids, either randomly or (preferably) using k-means++, which places the initial centroids far apart.
3. Assignment step — compute the Euclidean distance from every data point to each centroid and assign the point to the nearest one.
4. Update step — recompute each centroid as the arithmetic mean of all points currently assigned to that cluster.
5. Repeat the assignment and update steps until the centroids no longer move appreciably, the assignments stop changing, or the maximum number of iterations is reached.

The objective function minimised is $J = \Sigma_i \Sigma(x \in C_i) ||x - \mu_i||^2$. Since the objective decreases at every iteration and the number of possible assignments is finite, the algorithm always converges — though only to a local minimum, which is why it is usually run several times with different initialisations (n_init) and the best result is kept.

Q3. Why is feature scaling important before applying K-Means clustering?

K-Means uses Euclidean distance to decide cluster membership. Distance is dominated by the feature with the largest numeric range, so an unscaled feature effectively gets more weight in the clustering.

For example, if annual income is measured in rupees (say 3,00,000–20,00,000) and age in years (18–70), a difference of one lakh in income completely overwhelms a difference of thirty years in age. The resulting clusters would essentially be income bands and age would be ignored.

Standardisation ($z = (x - \mu) / \sigma$) or min–max normalisation puts every feature on a comparable scale so that each one contributes equally to the distance calculation. Scaling also helps the algorithm converge faster and makes the WCSS and silhouette values comparable across different feature sets.

Q4. What is the Elbow Method, and how does it help determine the optimal number of clusters?

The Elbow Method is a heuristic for choosing K. K-Means is run for a range of values of K (typically 1 to 10) and for each run the WCSS, also called inertia, is recorded. WCSS is the sum of squared distances of every point from its own centroid.

WCSS always decreases as K increases (with K = n it becomes zero), so the minimum value cannot be used directly. Instead, WCSS is plotted against K. Initially the curve drops steeply because each new cluster genuinely explains additional structure. After a certain point the extra clusters only split already-compact groups, so the curve flattens. The point where the steep fall changes into a gentle slope resembles the bend of an elbow, and that value of K is taken as optimal.

In this experiment the sharp bend occurred at K = 5. The method is simple and quick, but the elbow is sometimes ambiguous, which is why it should be confirmed with a quantitative measure such as the Silhouette Score.

Q5. What is the Silhouette Score? How is it used to evaluate clustering performance?

The Silhouette Score measures how well each point fits into its own cluster compared with the nearest neighbouring cluster. For a point i:

$$s(i) = (b(i) - a(i)) / \max(a(i), b(i))$$

where a(i) is the average distance from i to all other points in its own cluster (cohesion) and b(i) is the average distance from i to all points in the nearest other cluster (separation).

The value lies between −1 and +1:

- Close to +1 — the point is far from neighbouring clusters and is well clustered.
- Around 0 — the point lies on or very near the boundary between two clusters.
- Negative — the point has probably been assigned to the wrong cluster.

The average silhouette score over all points measures the quality of the whole clustering. Clustering is repeated for different values of K and the K giving the highest average score is chosen. In this experiment K = 5 gave the highest score of 0.5546, confirming the Elbow Method result.

Q6. Why is K-Means considered an unsupervised machine learning algorithm?

K-Means is unsupervised because it is trained on data that has no target variable and no ground-truth labels. The algorithm is never told which customer belongs to which group; it infers the groups purely from the similarity (distance) between the input feature vectors.

There is no teacher signal, no error computed against a correct answer, and no accuracy that can be measured against known labels. The cluster numbers it outputs (0, 1, 2 …) are arbitrary identifiers with no predefined meaning —

the analyst must interpret them afterwards. The goal is to describe the hidden structure of the data rather than to predict a known output, which is the defining characteristic of unsupervised learning.

Q7. Mention any four real-world applications of customer segmentation.

1. Targeted marketing in retail and e-commerce — grouping customers by purchasing power and behaviour so that offers, emails and advertisements are sent only to the segments most likely to respond, reducing marketing cost and increasing conversion.
2. Banking and financial services — segmenting account holders to offer suitable credit cards, loans and investment products, and to support credit-risk assessment and fraud detection.
3. Telecom churn management — identifying groups of subscribers with similar usage patterns so that the segment showing signs of churn can be given retention offers before it leaves.
4. Recommendation systems in streaming and online platforms — clustering users with similar tastes so that content watched or bought by one member of a cluster can be recommended to the others.

Other applications include personalised insurance premium design, market-basket-driven store layout planning, and segmentation of patients in healthcare for preventive care programmes.

Q8. What are the limitations of the K-Means algorithm?

- The number of clusters K must be specified in advance, and the correct value is not known beforehand.
- The final result depends on the initial placement of centroids; a poor initialisation can trap the algorithm in a local optimum. (k-means++ and multiple restarts reduce but do not eliminate this.)
- It assumes clusters are spherical, of roughly equal size and of similar density, so it performs badly on elongated, nested or irregularly shaped clusters.
- It is very sensitive to outliers, because a single extreme point can pull a centroid a long way — the mean is not a robust statistic.
- It requires feature scaling, since it depends on Euclidean distance.
- It works with numerical data only; categorical attributes need encoding or a variant such as K-Modes.
- Every point is forced into exactly one cluster (hard assignment), with no notion of a point partially belonging to two clusters or being noise.
- Performance degrades in very high-dimensional spaces because distances become less meaningful (the curse of dimensionality).

Q9. How can businesses use customer segmentation to improve marketing and customer retention?

Segmentation converts a single, generic strategy into several focused ones. In marketing it allows the business to:

- Send personalised offers, product recommendations and messages that match each group’s income level and buying habits, which raises the response and conversion rate.
- Allocate the advertising budget efficiently by spending more on high-value segments and less on segments with poor returns.
- Decide pricing, discount levels and product assortment for each group, and choose the right channel (social media for young impulsive buyers, email or in-store events for premium customers).

In retention it allows the business to:

- Identify high-value customers and protect them with loyalty programmes, VIP privileges and dedicated service.
- Detect segments showing declining activity — an early warning of churn — and target them with win-back campaigns before they leave.
- Understand the reason for low engagement in a particular segment (for instance the high-income/low-spending group) and address it directly.
- Improve customer lifetime value by gradually moving average customers into higher-value segments through cross-selling and upselling.

Q10. Compare K-Means Clustering with Hierarchical Clustering.

Both are unsupervised clustering techniques based on distance, but they differ in approach as shown below.

Basis	K-Means Clustering	Hierarchical Clustering
Approach	Partitional — divides data into K flat clusters at once	Builds a nested hierarchy of clusters (agglomerative or divisive)
Number of clusters	K must be fixed before running	Not required in advance; chosen afterwards by cutting the dendrogram
Output	Cluster labels and centroids	A dendrogram showing the merge/split sequence
Time complexity	Approximately $O(n \cdot K \cdot i \cdot d)$ — fast and scalable	$O(n^2 \log n)$ with $O(n^2)$ memory — slow on large data
Reassignment	Points can move between clusters across iterations	Once merged or split, the decision is irreversible
Cluster shape	Works best for spherical, similarsized clusters	Can capture non-spherical and nested structures depending on linkage
Outlier sensitivity	High — centroids are pulled by outliers	Moderate; single-linkage is sensitive, Ward linkage more robust
Reproducibility	Depends on initialisation; may differ between runs	Deterministic for a given linkage and distance metric
Best suited for	Large datasets, customer segmentation, image compression	Smaller datasets where the relationship between groups matters, e.g. gene expression, taxonomy

Result: The K-Means clustering algorithm was implemented successfully and the customers were segmented into five meaningful groups. The clustering was validated using the Elbow Method and the Silhouette Score.